01 / LA PREGUNTA FÍSICA

¿Cómo nace una onda de choque?

Partimos del LPMD que ya instalaste y usaste para simular argón cristalino.

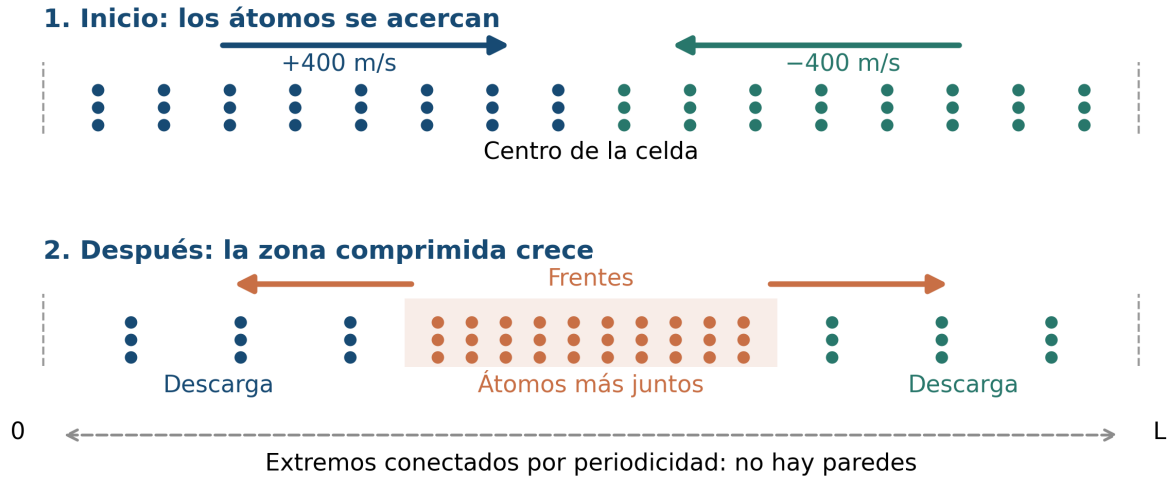
Dos preguntas para la clase

Física: ¿qué ocurre cuando dos partes de un cristal se encuentran?

Cómputo: ¿cuánto tiempo ahorramos al repartir el cálculo entre varios hilos?

Cómo se produce la compresión

Esquema, no a escala



Al comenzar
Cada mitad del cristal viaja hacia la otra a 400 m/s.

Después del encuentro
En el centro los átomos quedan más juntos y el flujo se frena.

Predice antes de ejecutar

En el centro, la densidad **aumentará / disminuirá**. ¿Qué esperas de la velocidad media?

Actualiza el repositorio antes de buscar el ejemplo

Abre una terminal en la raíz del repositorio que clonaste y ejecuta:

```
git pull --ff-only
```

Así descargas los cambios del código y los ejemplos. El programa instalado se actualiza al recompilar y reinstalar, como se indica en la página 2.

Prepara tu carpeta de trabajo con el administrador de archivos

Después de actualizar, copia `lpmd/examples/shock-wave` a un lugar de tu preferencia. Dentro de tu copia, crea tres carpetas: `prueba`, `choque` y `tiempos`. Conserva los scripts junto a ellas.
Mi carpeta de trabajo: _____
Abre las terminales con *Abrir en terminal* en la carpeta indicada en cada paso. Así cada estudiante puede usar sus propias rutas.

02 / COMPROBAR Y ACTUALIZAR LA INSTALACIÓN

¿Tu instalación tiene OpenMP?

Comprueba el programa instalado e incorpora los cambios descargados con Git.

A Haz una prueba corta con tu instalación

Copia `smoke.control` dentro de `prueba` y abre una terminal allí.

```
command -v lpmd
OMP_NUM_THREADS=2 OMP_DYNAMIC=FALSE OMP_DISPLAY_ENV=TRUE \
  lpmd smoke.control > run.log 2>&1
```

El primer comando muestra qué ejecutable usas. Anota su ruta:

El segundo pide dos hilos, evita que OpenMP ajuste ese número y guarda los mensajes en `run.log`. Ábrelo con un editor de texto y busca:

`SIMULATION FINISHED`: la prueba terminó.
`OPENMP DISPLAY ENVIRONMENT` y `_OPENMP`: se cargó soporte OpenMP.
`OMP_NUM_THREADS = '2'`: se solicitaron dos hilos.

Si descargaste cambios del código o falta OpenMP, sigue el paso B. Si ya reinstalaste ese código y aparecen las tres señales, continúa en la página 3. Si hubo un error de simulación, revísalo con el docente antes de continuar.

B Recompila e instala el código actualizado

Busca la carpeta donde compilaste **este código de lpmd2**: puede llamarse `build` u otro nombre. Debe contener `CMakeCache.txt`. Ábrelo y comprueba que `CMAKE_INSTALL_PREFIX` corresponde a la instalación anotada arriba; por ejemplo, `/usr/local` para `/usr/local/bin/lpmd`. Estos comandos usan `Unix Makefiles`; comprueba el campo `CMAKE_GENERATOR` del mismo caché. Si usaste otro generador, consulta el README antes de continuar.

Abre una terminal **en esa carpeta de compilación** y ejecuta, una línea a la vez:

```
cmake . -DLPMD_ENABLE_OPENMP=ON -DCMAKE_BUILD_TYPE=Release
cmake --build . --target preinstall --parallel 4
sudo cmake --install .
```

El punto significa *esta carpeta*; se conservan las rutas registradas por CMake. La primera línea activa OpenMP. La segunda compila y prepara la instalación, incluidas las copias `CMakeRelink.dir` cuando hacen falta; espera a que termine sin errores. La tercera reinstala los archivos preparados. **Compilar con `--parallel 4` y simular con cuatro hilos son decisiones distintas.**

Vuelve a comprobar

Regresa a `prueba` y repite el paso A. Debe aparecer OpenMP y terminar la corrida. Si la terminal encuentra otra copia de LPMD, usa la ruta completa del recién instalado en lugar de `lpmd`. Si falta el caché o las rutas no coinciden, revisa tu instalación con el docente.

03 / EJECUTAR Y GRAFICAR

Observar el choque

Copia `shock.control` a la carpeta `choque` y abre una terminal allí.

6912 átomos de Ar	**4000 pasos**	**2 ps físicos**
Cristal ordenado FCC	0.5 fs por paso	$1\ \mathrm{ps} = 1000\ \mathrm{fs}$

```
OMP_NUM_THREADS=4 OMP_DYNAMIC=FALSE \
  lpmd shock.control > run.log 2>&1
```

Usa 1, 2 o 4 hilos según los núcleos disponibles para tu trabajo. Espera a que vuelva el indicador de la terminal y confirma `SIMULATION FINISHED` en el registro. Se crean `shock.xyz`, `shock-monitor.dat` y `shock-final.dump`.

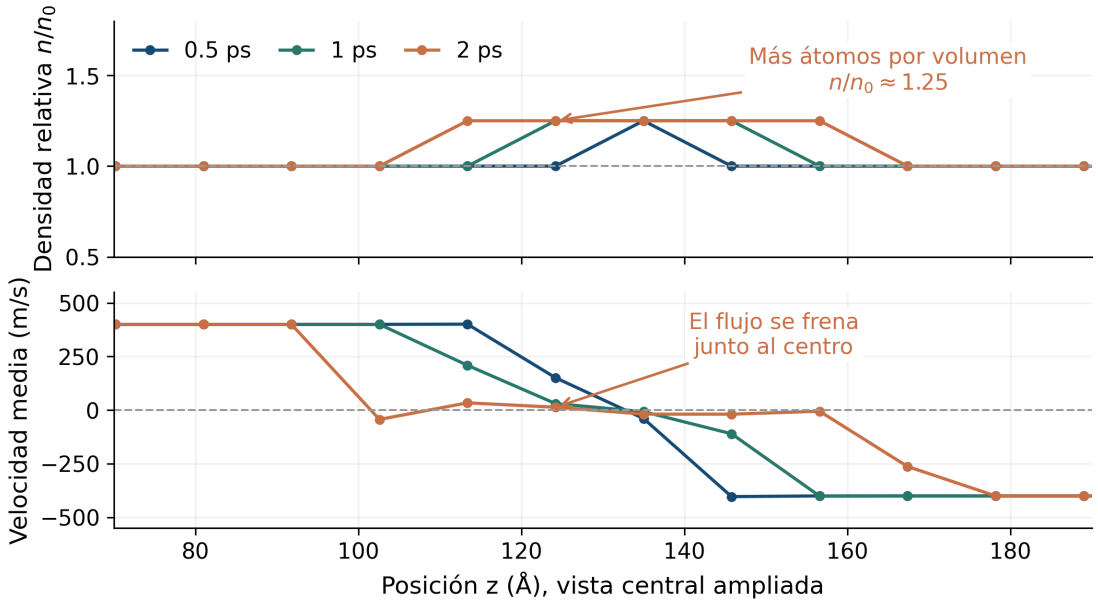
Transforma la trayectoria en dos gráficos

En la misma terminal, ejecuta estas dos líneas. Python se usa aquí para agrupar los átomos por posición y dibujar tus resultados:

```
python3 ../profiles.py shock.xyz --output perfiles.csv
python3 ../plot_results.py perfiles.csv
```

Abre `shock-profiles.png` con doble clic. Necesitas Python 3 y Matplotlib; si falta este último, sigue la instalación indicada en el README.

Datos reales: acercamiento a la zona central



Referencia calculada con este ejemplo, ampliada alrededor del centro. Tu figura muestra toda la celda. $n/n_0 > 1$ indica compresión; una velocidad media cercana a cero indica que el flujo se frenó. Busca cómo crece esa zona.

04 / MEDIR EL TIEMPO DE ESPERA

Comparar 1, 2 y 4 hilos

Mantén el mismo sistema. Cambia únicamente el número de hilos solicitados.



Reparto ideal. En el programa real también hay coordinación y escritura.

A Prepara una comparación breve

Dentro de `tiempos`, crea `1-hilo`, `2-hilos` y `4-hilos`. Copia `tiempos.control` en cada una. Este control conserva los 6912 átomos y el paso de 0.5 fs; usa **200 pasos** y omite XYZ en todos los casos. Así la medición dura menos y escribe menos datos.

B Ejecuta desde cada carpeta

Abre una terminal en la carpeta correspondiente y usa su línea:

```
OMP_NUM_THREADS=1 OMP_DYNAMIC=FALSE lpmd tiempos.control > run.log 2>&1
OMP_NUM_THREADS=2 OMP_DYNAMIC=FALSE lpmd tiempos.control > run.log 2>&1
OMP_NUM_THREADS=4 OMP_DYNAMIC=FALSE lpmd tiempos.control > run.log 2>&1
```

Haz tres corridas por número de hilos. Tras cada una, abre `run.log`, confirma `SIMULATION FINISHED` y anota `Elapsed real time`. Desde el administrador de archivos, renombra el registro como `r1.log`, `r2.log` o `r3.log` antes de repetir la línea. Las otras salidas de esta comparación se reemplazan.

Hilos	Corrida 1 (s)	Corrida 2 (s)	Corrida 3 (s)	Mediana (s)
1				
2				
4				

Compara con justicia

La **mediana** es el valor del medio tras ordenar los tres tiempos. Usa **real**, que mide espera; *user* y *system* suman tiempo de CPU. Deja terminar una corrida antes de iniciar otra y evita otros cálculos simultáneos. Si dispones de menos de cuatro núcleos, mide solo las filas que puedas usar.

05 / INTERPRETAR

¿Qué aprendemos de los resultados?

Trabaja con tus perfiles y con las medianas que acabas de medir.

1 Relaciona densidad y movimiento

En tu gráfico, a 2 ps y cerca del centro, leo aproximadamente:

$n/n_0 =$ _____ velocidad media = _____ m/s.

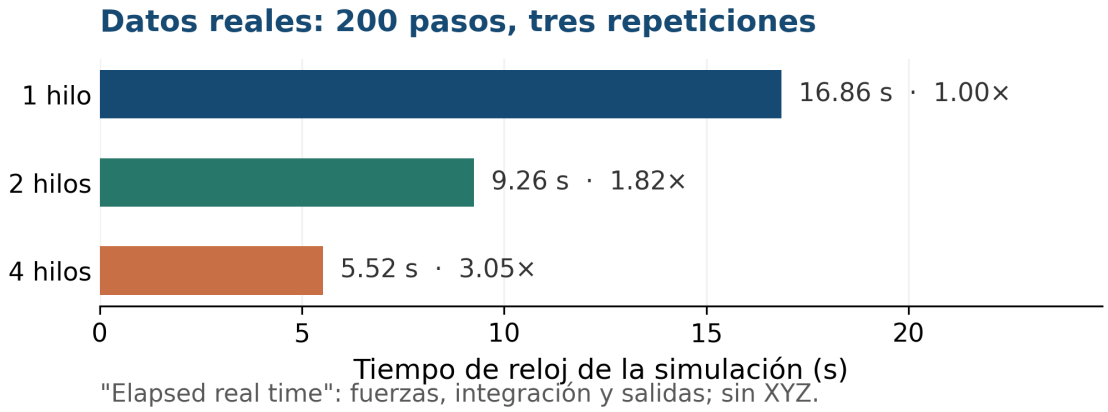
¿Coincide con tu predicción? ¿Cómo cambia el ancho de la zona comprimida?

2 Calcula cuánto se acelera el programa

$$S_p = \frac{\text{mediana con 1 hilo}}{\text{mediana con } p \text{ hilos}}.$$

Con 2 hilos: $S_2 =$ _____ Con 4 hilos: $S_4 =$ _____

Si $S = 2$, la simulación tarda la mitad. ¿La mejora es proporcional al número de hilos? Da una posible razón para lo que observaste.



"Elapsed real time": fuerzas, integración y salidas; sin XYZ.

Referencia de un equipo: medianas de tres corridas de 200 pasos, leyendo `Elapsed real time` como en la página 4. Tus tiempos pueden ser distintos.

3 Distingue los dos relojes

El choque completo representa **2 ps físicos**. ¿Usar más hilos cambia esos 2 ps o cambia lo que esperas frente al computador?

Entrega breve

Entrega tu gráfico de perfiles, la tabla de tiempos y estas tres respuestas. Indica procesador, número de hilos y ruta del ejecutable usado. Conserva los registros para poder revisar la comparación.

Para interpretar: el modelo frío y periódico produce compresión central y descarga en los extremos. La temperatura global incluye el movimiento colectivo; no mide por sí sola el calentamiento local.